



Institución
Universitaria
Reacreditada en Alta Calidad

ESTRUCTURA DE DATOS Y »»» LABORATORIO «««

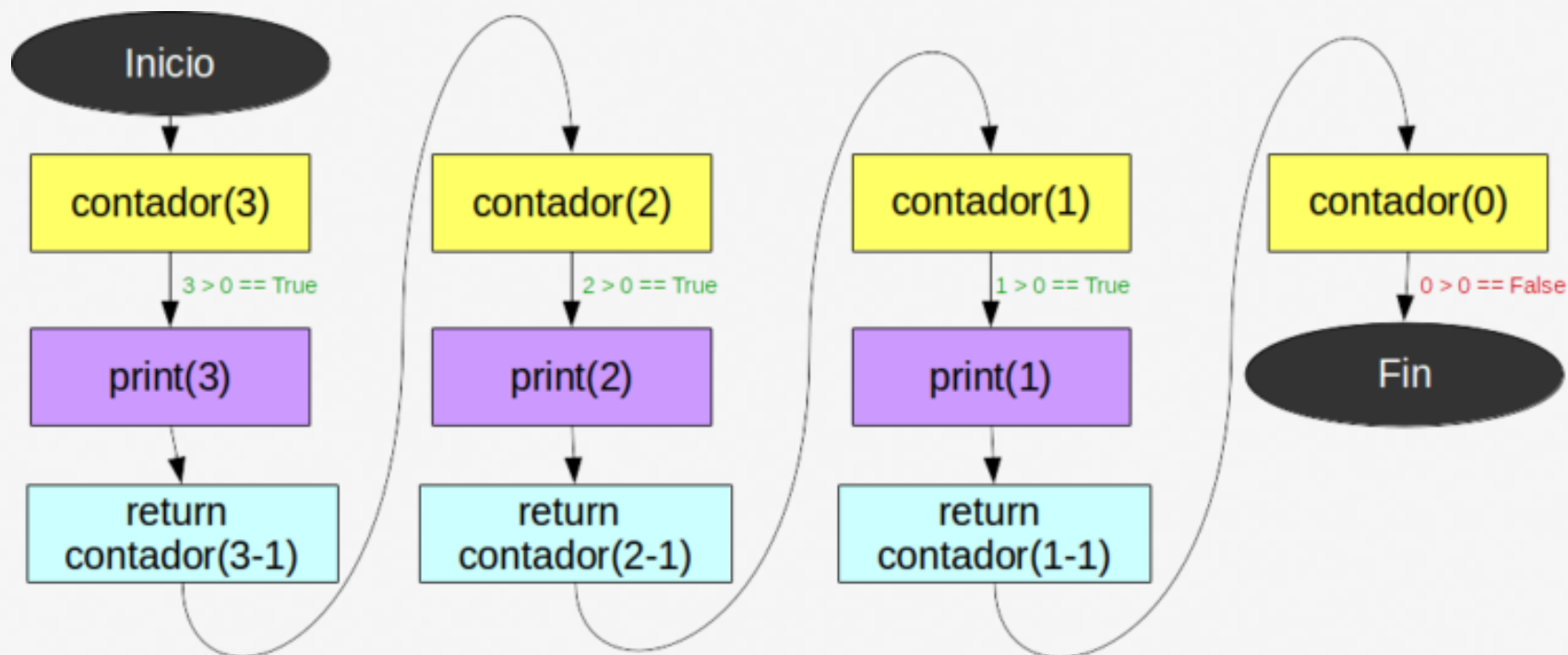
580506004-8

William Giraldo Escobar
Facultad de Ingenierías
Departamento de Sistemas de Información



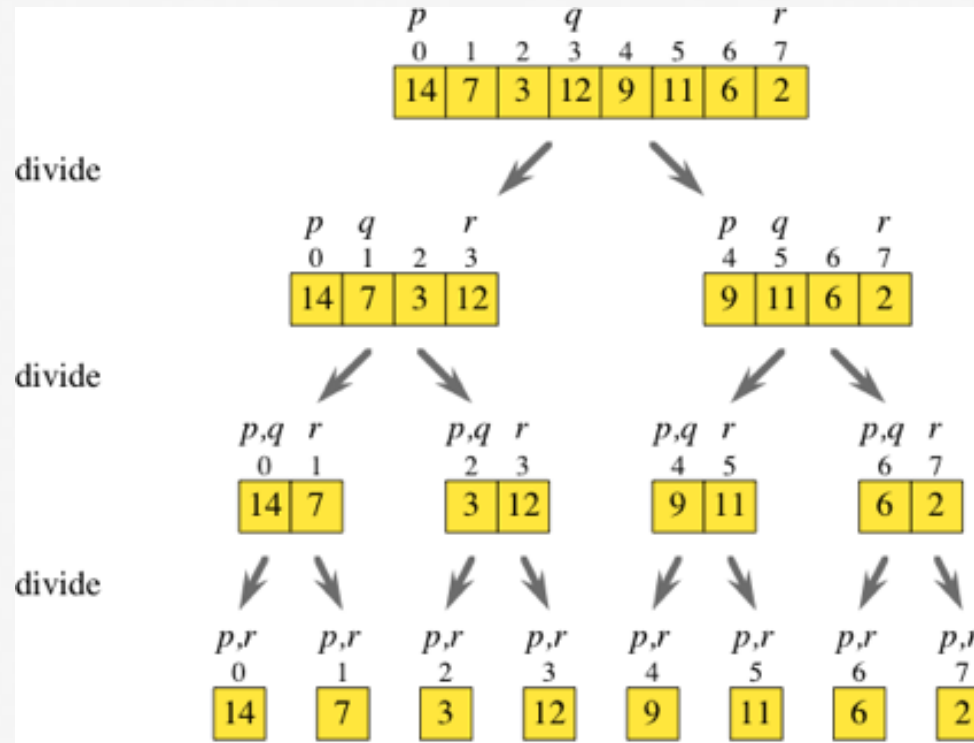
ALGORITMOS RECURSIVOS

La *recursión* es un enfoque en programación en el que una *función se llama a sí misma* para resolver un problema más grande. *Reemplaza* en ocasiones el uso de ciclos.



ALGORITMOS RECURSIVOS

En el contexto de algoritmos, se utiliza para *dividir* un problema en subproblemas más *pequeños* y resolverlos de manera similar.



ALGORITMOS RECURSIVOS

Para que una función sea recursiva, debe tener dos partes clave:

- *Caso base*: Es el caso en el que la *recursión se detiene*. Cuando la función alcanza el caso base, simplemente devuelve un valor sin hacer llamadas recursivas adicionales.
- *Caso recursivo*: Es donde la función *se llama a sí misma* con un conjunto de argumentos diferentes. La idea es que cada llamada recursiva resuelva una parte más pequeña del problema hasta llegar al caso base.

ALGORITMOS RECURSIVOS

Un algoritmo recursivo se llama a sí mismo, si no se define una condición de parada (caso base), se ejecutará infinitamente

```
def repetir():  
    repetir()
```



Caso recursivo

ALGORITMOS RECURSIVOS

```
def imprimir(x):  
    print(x)  
    imprimir(x-1)  
  
imprimir(5)
```

5 4 3 2 1 0 -1 -2 -3 . . .

→ *Caso recursivo*

ALGORITMOS RECURSIVOS

Siempre se debe definir un *caso base*

```
def imprimir(x):
```

```
    if x>0:
```

```
        print(x)
```

```
        imprimir(x-1)
```

Caso base

Caso recursivo

ALGORITMOS RECURSIVOS

```
def imprimir(x):  
    if x>0:  
        print(x)  
        imprimir(x-1)
```

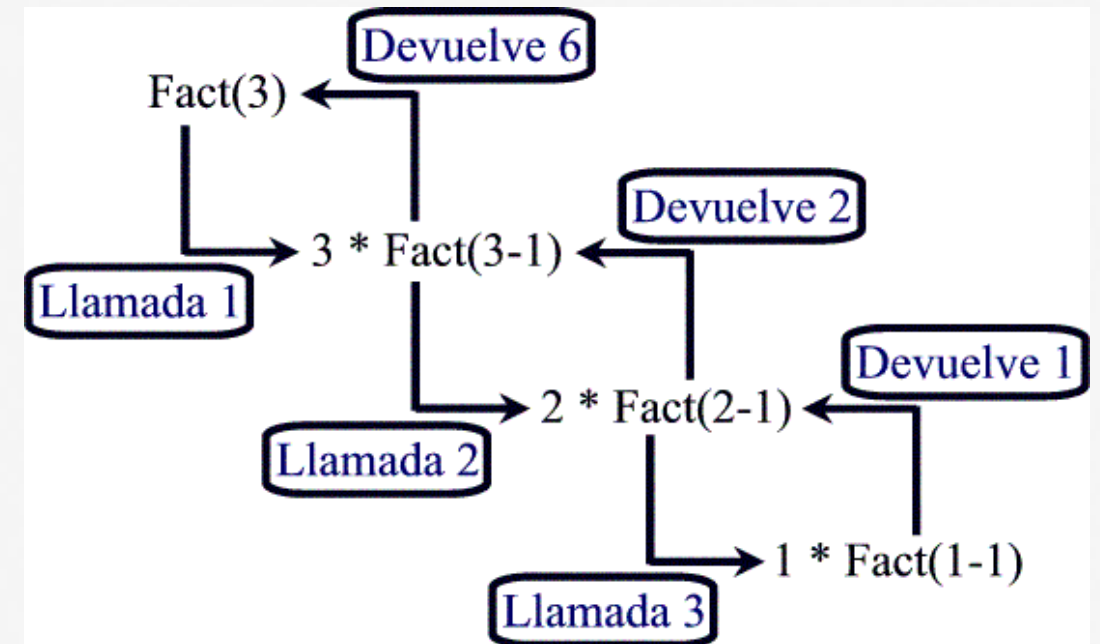
`imprimir(5)`

5	←	print(5)
4	←	Imprimir(4 – 1)
3	←	Imprimir(3 – 1)
2	←	Imprimir(2 – 1)
1	←	Imprimir(1 – 1) -> caso base

ALGORITMOS RECURSIVOS

Ejemplo clásico : calcular el factorial

```
def factorial(n):  
    # Caso base: cuando n es igual a 0 o 1  
    if n == 0 or n == 1:  
        return 1  
    # Caso recursivo:  $n! = n * (n-1)!$   
    else:  
        return n * factorial(n - 1)
```



ALGORITMOS RECURSIVOS

Calcular una potencia: $2^3 = 2 * 2 * 2$

```
def potencia(base, exponente):  
    # Caso base  
    if exponente == 0:  
        return 1  
    # Caso recursivo  
    else:  
        return base * potencia(base, exponente - 1)
```

ALGORITMOS RECURSIVOS

Sumar los elementos sucesivos desde un número n hasta un número m

$$2 + 3 + 4 + 5$$

```
def suma_desde_hasta(inicio, fin):  
    # Caso base: Si el inicio es igual al fin, retorna ese valor  
    if inicio == fin:  
        return inicio  
    # Caso recursivo  
    else:  
        return inicio + suma_desde_hasta(inicio + 1, fin)
```



Institución
Universitaria
Reacreditada en Alta Calidad

ACTIVIDAD

- Cree un algoritmo recursivo que imprima los números impares entre los valores n y m
- Cree un algoritmo recursivo que sume los n números pares

ALGORITMOS RECURSIVOS Y LISTAS

```
def suma_lista(lista):  
    # Caso base: Si la lista está vacía, la suma es 0.  
    if not lista:  
        return 0  
    # Caso recursivo: Suma el primer elemento con la suma de los demás  
    else:  
        return lista[0] + suma_lista(lista[1:])
```

[1, 2, 3, 4, 5] → [2, 3, 4, 5] → [3, 4, 5]

Actividad

- Cree una función recursiva `maximo(lista)` que encuentre y devuelva el mayor elemento de una lista (sin usar `max()`)
- Cree una función recursiva `replicar(lista, n)` para replicar los elementos de una lista una cantidad `n` de veces. Por ejemplo:
`replicar([1, 3, 4, 7], 2) -> ([1, 1, 3, 3, 4, 4, 7, 7])`



Institución
Universitaria
Reacreditada en Alta Calidad

¡MUCHAS GRACIAS!

A decorative graphic consisting of several horizontal lines in various colors (yellow, orange, red, purple, blue, green) and a curved line on the right side, positioned below the main text.